

Cost of Living Experimentation Platform

End-to-End Data Pipeline Documentation

PySpark · PostgreSQL · FastAPI · Dagster · A/B Testing

Author	Idhaya Bastine Kennedy
Dataset	EPI Family Budget Calculator (~3,000 US Counties)
Language	Python 3.11
Date	May 2026

Table of Contents

1	Project Overview & Architecture
2	Dataset
3	Technology Stack
4	Ingest Layer — <code>pipeline/ingest.py</code>
5	Transform Layer — <code>pipeline/transform.py</code>
6	Load Layer — <code>pipeline/load.py</code>
7	A/B Experiment Engine — <code>experiments/ab_engine.py</code>
8	Orchestration — <code>orchestration/etl_job.py</code>
9	REST API — <code>api/main.py</code>
10	Local Demo Runner — <code>run_demo.py</code>
11	How To See the Output

1. Project Overview & Architecture

This project is a production-grade data engineering platform that ingests US county-level cost-of-living data, processes it through a PySpark ETL pipeline, stores results in PostgreSQL, runs statistical A/B experiments across US Census regions, and exposes all results through a self-documenting FastAPI REST interface.

Architecture Flow



PostgreSQL Tables

Table	Contents
cost_of_living_processed	Cleaned county-level records (~3,000 rows)
regional_cost_statistics	Spark SQL aggregates per US Census region (5 rows)
experiment_results	Pairwise A/B test results (p-value, Cohen's d, CI, winner)

2. Dataset

Source: Economic Policy Institute (EPI) Family Budget Calculator — cost_of_living_us_dap2.xls

The dataset covers approximately 3,000 US counties and metropolitan areas, providing annual cost estimates across nine expenditure categories for a family of four (2 adults, 2 children).

Column	Type	Description
case_id	Integer	Unique row identifier
state	String	US state abbreviation (e.g. CA)
areaname	String	Metropolitan/micropolitan area name
county	String	County name
housing_cost	Float	Annual housing cost (USD)
food_cost	Float	Annual food cost (USD)
transportation_cost	Float	Annual transportation cost (USD)
healthcare_cost	Float	Annual healthcare cost (USD)
other_necessities_cost	Float	Other annual necessities (USD)
childcare_cost	Float	Annual childcare cost (USD)
taxes	Float	Annual taxes (USD)
total_cost	Float	Total annual cost of living (USD)
median_family_income	Float	Median annual family income (USD)

3. Technology Stack

Layer	Technology	Version	Why
Batch Processing	PySpark	3.5.1	Mirrors Databricks DataFrame API; Catalyst optimiser; partitioned Parquet
Orchestration	Dagster	1.7.6	Type-validated op outputs, run history, lineage — same role as Databricks
Analytical Store	PostgreSQL	15+	ACID writes; complex SQL joins for the API layer
Experimentation	SciPy	1.12.0	Welch's t-test — correct for unequal group variances across US regions
API	FastAPI	0.111.0	Auto-generated OpenAPI docs; async-ready; Pydantic validation
Data Wrangling	Pandas	2.1.4	XLS ingestion bridge and pandas→SQL via SQLAlchemy
Containers	Docker + Compose	latest	Reproducible full-stack: PostgreSQL + API in one docker-compose up
DB ORM	SQLAlchemy	2.0.30	DB-agnostic engine; works with both PostgreSQL (prod) and SQLite (dev)

4. Ingest Layer — pipeline/ingest.py

The ingest layer is responsible for reading the raw CSV-formatted XLS source file, enforcing a strongly typed schema, and writing the data to a Parquet staging zone. PySpark is used here to mirror the Databricks/production execution environment.

Key Concepts

- **SparkSession** — entry point to all PySpark functionality; configured with adaptive query execution and 8 shuffle partitions for local demo efficiency.
- **StructType schema enforcement** — every column has an explicit type. Without this, Spark would infer types and may silently coerce floats to strings.
- **Pandas bridge** — the XLS file is tab/comma-delimited without a native Spark connector, so pandas reads it first, then hands the DataFrame to Spark.
- **Parquet output** — columnar format; far more efficient for the downstream column-selective GROUP BY queries than row-based CSV.

pipeline/ingest.py

```
"""
Ingestion layer – reads raw XLS/CSV source into Parquet staging zone.
Uses PySpark for schema enforcement and scalable I/O.
"""

import pandas as pd
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.types import (
    StructType, StructField, StringType, FloatType, IntegerType
)

RAW_SCHEMA = StructType([
    StructField("case_id", IntegerType(), True),
    StructField("state", StringType(), True),
    StructField("areaname", StringType(), True),
    StructField("county", StringType(), True),
    StructField("isMetro", StringType(), True),
    StructField("housing_cost", FloatType(), True),
    StructField("food_cost", FloatType(), True),
    StructField("transportation_cost", FloatType(), True),
    StructField("healthcare_cost", FloatType(), True),
    StructField("other_necessities_cost", FloatType(), True),
    StructField("childcare_cost", FloatType(), True),
    StructField("taxes", FloatType(), True),
    StructField("total_cost", FloatType(), True),
    StructField("median_family_income", FloatType(), True),
])

def get_spark() -> SparkSession:
    return (
        SparkSession.builder
        .appName("CostOfLivingPipeline")
        .master("local[*]")
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
        .getOrCreate()
    )

def ingest(source_path: str, staging_path: str = "data/staging") -> int:
```


5. Transform Layer — pipeline/transform.py

The transform layer reads staged Parquet data and applies a sequence of enrichment and cleaning operations. The output is partitioned Parquet, structured for efficient predicate pushdown on region-level queries.

Operations Performed

- **Broadcast join** — expands state abbreviations (AL → Alabama) using a tiny lookup DataFrame that Spark broadcasts to all executors, avoiding a shuffle.
- **String normalisation** — county and areaname fields contain embedded underscores (e.g. 'Jefferson_County'); split on '_' and keep the first token.
- **Null fill** — median_family_income NULLs are replaced with 0.0 so affordability_ratio never produces NaN.
- **Deduplication** — dropDuplicates(['county']) removes any repeated entries caused by multiple family-type variants in the source dataset.
- **Derived column: affordability_ratio** — median_family_income / total_cost. A value > 1.0 means the median income exceeds the cost of living.
- **Derived column: region** — US Census region label built with a PySpark when/otherwise chain (equivalent to a SQL CASE expression).
- **Partitioned Parquet output** — writing partitionBy('region') means queries like WHERE region = 'West' skip all other partitions entirely.

pipeline/transform.py

```
"""
Transform layer – cleans, enriches, and partitions cost-of-living data.

Key operations:
- Expand US state abbreviations via broadcast join
- Normalise county/areaname strings
- Fill nulls and deduplicate
- Add derived metrics: affordability_ratio, region label
- Partition output by region for predicate-pushdown efficiency
"""
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql import functions as F
from pipeline.ingest import get_spark

STATE_ABBREV = {
    'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR': 'Arkansas',
    'CA': 'California', 'CO': 'Colorado', 'CT': 'Connecticut', 'DE': 'Delaware',
    'DC': 'District of Columbia', 'FL': 'Florida', 'GA': 'Georgia', 'HI': 'Hawaii',
    # ... (all 50 states + territories)
}

REGION_MAP = {
    'Northeast': ['Maine', 'New Hampshire', 'Vermont', 'Massachusetts', 'Rhode Island',
                  'Connecticut', 'New York', 'New Jersey', 'Pennsylvania'],
    'Southeast': ['Maryland', 'Delaware', 'Virginia', 'West Virginia', 'North Carolina',
                  'South Carolina', 'Georgia', 'Florida', 'Alabama', 'Mississippi',
                  'Tennessee', 'Kentucky'],
    'Midwest': ['Ohio', 'Michigan', 'Indiana', 'Wisconsin', 'Illinois', 'Minnesota',
                'Iowa', 'Missouri', 'North Dakota', 'South Dakota', 'Nebraska', 'Kansas'],
    'Southwest': ['Texas', 'Oklahoma', 'New Mexico', 'Arizona'],
    'West': ['Colorado', 'Wyoming', 'Montana', 'Idaho', 'Washington', 'Oregon',
```


6. Load Layer — pipeline/load.py

The load layer moves processed data from Parquet into PostgreSQL. Two tables are populated: county-level detail records and regional aggregates computed via Spark SQL before the data leaves the cluster.

Key Concepts

- **toPandas() + SQLAlchemy** — avoids needing a JDBC driver for the local/demo run. In production this would be swapped for a Spark JDBC write or Delta Lake write.
- **Spark SQL aggregation before collect** — the GROUP BY runs distributed on Spark before toPandas(); only 5 summary rows are collected, not 3,000.
- **chunksize=500** — Pandas writes rows to PostgreSQL in batches of 500 to avoid a single large INSERT that could exhaust server memory.
- **if_exists='replace'** — makes the pipeline fully idempotent; re-running overwrites previous results without duplication.

pipeline/load.py

```
"""
Load layer – writes processed Parquet data to PostgreSQL.

Uses toPandas() + SQLAlchemy for the DB write so no JDBC driver is needed
for local/demo runs. In production this would be swapped for a Spark JDBC
write with a connection pool or a Databricks Delta Lake target.
"""
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql import functions as F
from sqlalchemy import create_engine
from pipeline.ingest import get_spark

def load(
    processed_path: str,
    db_url: str = "postgresql://dap:dap@127.0.0.1:5432/dap",
) -> int:
    """
    1. Write county-level processed records to `cost_of_living_processed`
    2. Compute regional aggregates in Spark SQL → `regional_cost_statistics`
    Returns the number of county-level rows loaded.
    """
    spark = get_spark()
    engine = create_engine(db_url)

    df: DataFrame = spark.read.parquet(processed_path)
    df.createOrReplaceTempView("cost_of_living")

    # County-level load
    county_pd = df.toPandas()
    county_pd.to_sql(
        "cost_of_living_processed", engine,
        if_exists="replace", index=False, chunksize=500,
    )
    print(f"[load] {len(county_pd):,} county rows → cost_of_living_processed")

    # Regional aggregates via Spark SQL (heavy computation stays on Spark)
    regional_df: DataFrame = spark.sql("""
        SELECT
            region,
```


7. A/B Experiment Engine — experiments/ab_engine.py

The experiment engine applies rigorous statistical testing to compare cost-of-living metrics between any two US Census regions. This mirrors the kind of work a Business Experimentation & Optimisation team does — building infrastructure that lets analysts ask 'did this difference have a statistically significant effect?'

Statistical Methodology

Concept	Formula / Explanation
Hypothesis	H_0 : $\text{mean}(\text{metric}, A) = \text{mean}(\text{metric}, B)$ H_A : means differ (two-tailed)
Test	Welch's t-test (<code>ttest_ind</code> , <code>equal_var=False</code>) — correct when group variances differ
Significance	$p_value < \alpha$ (default $\alpha = 0.05$)
Effect Size	Cohen's $d = (\text{treatment_mean} - \text{control_mean}) / \text{pooled_std}$ Small < 0.2 Medium < 0.5 Large ≥ 0.8
Confidence Interval	95% CI on mean difference using Welch–Satterthwaite degrees of freedom $CI = \text{mean_diff} \pm t_crit \times SE_diff$
Lift %	$(\text{treatment_mean} - \text{control_mean}) / \text{control_mean} \times 100$
Winner	For cost metrics: lower is better. For affordability_ratio: higher is better.

experiments/ab_engine.py

```
"""
A/B Experimentation Engine

Implements the statistical core of an experimentation platform:
- Welch's t-test (correct when group variances differ)
- Cohen's d effect size
- 95% confidence interval on the mean difference
- Lift % and winner determination
"""
from __future__ import annotations
import uuid
from dataclasses import dataclass, asdict
from typing import Literal
import numpy as np
import pandas as pd
from scipy import stats
from sqlalchemy import Engine, text

VALID_REGIONS = {"Northeast", "Southeast", "Midwest", "Southwest", "West"}
VALID_METRICS = {
    "total_cost", "housing_cost", "food_cost",
    "healthcare_cost", "childcare_cost", "transportation_cost",
    "affordability_ratio",
}

@dataclass
```


8. Orchestration — orchestration/etl_job.py

Dagster orchestrates the pipeline by wrapping each stage as a typed op. Ops pass their return values (record counts) downstream, giving Dagster type-safe lineage tracking. The entire pipeline executes with a single dagster job execute command.

Why Dagster?

- **Typed op outputs** — each op declares its output type (`Out(int)`), and Dagster validates it before passing to the next op.
- **Run history & lineage** — every run is logged with timestamps, durations, and asset lineage in the Dagster UI.
- **Environment-driven config** — `SOURCE_PATH`, `DATABASE_URL`, etc. are injected via env vars, making the same job run locally and in production.
- **Same role as Databricks Workflows** — if this were deployed on Databricks, the `@job` decorator would be replaced by a Workflow definition with the same op order.

orchestration/etl_job.py

```
"""
Dagster job that orchestrates the full pipeline:
  ingest -> transform -> load -> run_experiments
"""
import os
from dagster import job, op, Out, In, get_dagster_logger

DB_URL = os.getenv("DATABASE_URL", "postgresql://dap:dap@127.0.0.1:5432/dap")
SOURCE_PATH = os.getenv("SOURCE_PATH", "cost_of_living_us_dap2.xls")
STAGING_PATH = os.getenv("STAGING_PATH", "data/staging")
PROCESSED_PATH = os.getenv("PROCESSED_PATH", "data/processed")

logger = get_dagster_logger()

@op(out=Out(int))
def ingest_op() -> int:
    from pipeline.ingest import ingest
    count = ingest(SOURCE_PATH, STAGING_PATH)
    logger.info("Ingested %d records", count)
    return count

@op(ins={"ingested": In(int)}, out=Out(int))
def transform_op(ingested: int) -> int:
    from pipeline.transform import transform
    count = transform(STAGING_PATH, PROCESSED_PATH)
    logger.info("Transformed %d records", count)
    return count

@op(ins={"transformed": In(int)}, out=Out(int))
def load_op(transformed: int) -> int:
    from pipeline.load import load
    count = load(PROCESSED_PATH, DB_URL)
    logger.info("Loaded %d records to PostgreSQL", count)
    return count

@op(ins={"loaded": In(int)}, out=Out(bool))
```


9. REST API — api/main.py

FastAPI serves the pipeline outputs over HTTP. Every endpoint is automatically documented at /docs (Swagger UI) and /redoc. Pydantic validates request bodies at the boundary, and SQLAlchemy handles parameterised queries to prevent SQL injection.

Method	Path	Description
GET	/health	Health check — returns {status: ok}
GET	/api/v1/regions	Regional cost statistics (5 rows)
GET	/api/v1/states?limit=N	Per-state aggregates, ordered by metric
GET	/api/v1/experiments	Experiment results; filter by metric/significance
POST	/api/v1/experiments/run	Run a new pairwise experiment on demand

api/main.py

```
"""
Cost of Living Experimentation Platform — REST API
Self-documenting at /docs (Swagger UI).
"""
import os
from typing import Literal
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel
from sqlalchemy import create_engine, text

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://dap:dap@127.0.0.1:5432/dap")
engine = create_engine(DATABASE_URL, pool_pre_ping=True)

app = FastAPI(
    title="Cost of Living Experimentation Platform",
    version="1.0.0",
)

VALID_REGIONS = {"Northeast", "Southeast", "Midwest", "Southwest", "West"}

class ExperimentRequest(BaseModel):
    control_region: str
    treatment_region: str
    metric: str = "total_cost"
    alpha: float = 0.05

@app.get("/health", tags=["ops"])
def health():
    return {"status": "ok"}

@app.get("/api/v1/regions", tags=["analytics"])
def get_regional_stats():
    with engine.connect() as conn:
        rows = conn.execute(
            text("SELECT * FROM regional_cost_statistics ORDER BY avg_total_cost DESC")
        ).mappings().all()
    return [dict(r) for r in rows]

@app.get("/api/v1/states", tags=["analytics"])
```


10. Local Demo Runner — run_demo.py

The demo runner replicates the full pipeline logic using pandas and SQLite — no Java, no PostgreSQL, no Docker required. It is ideal for quickly verifying end-to-end behaviour or for demos on a laptop. The production pipeline (pipeline/ + orchestration/) uses the identical algorithms on PySpark + PostgreSQL.

run_demo.py (core logic — mirrors all 4 pipeline steps)

11. How To See the Output

Option A — Run the Local Demo (No Docker, No Java needed)

Step 1: Install dependencies

Terminal

```
pip install -r requirements.txt
```

Step 2: Run the pipeline demo

Terminal

```
python3 run_demo.py
```

This prints the 4-step pipeline output to your terminal: record counts, region distribution, regional cost table, and A/B experiment results.

Step 3: Start the REST API

Terminal

```
DATABASE_URL=sqlite:///cost_of_living_demo.db uvicorn api.main:app --reload
```

The API starts at <http://localhost:8000>

Step 4: Open the interactive API docs

Browser

```
http://localhost:8000/docs
```

This opens the Swagger UI where you can click any endpoint and try it live.

Option B — curl the API endpoints

Terminal — example API calls

```
# Regional cost statistics
curl http://localhost:8000/api/v1/regions

# Top 10 most expensive states by total cost
curl "http://localhost:8000/api/v1/states?limit=10&order_by=total_cost"

# All significant experiment results for housing cost
curl "http://localhost:8000/api/v1/experiments?significant_only=true&metric=housing_cost"

# Run a new A/B experiment on demand (Northeast vs Midwest on housing cost)
curl -X POST http://localhost:8000/api/v1/experiments/run \
  -H "Content-Type: application/json" \
  -d '{"control_region": "Midwest", "treatment_region": "Northeast", "metric": "housing_cost"}'

# Health check
curl http://localhost:8000/health
```

Option C — Full Production Stack (Docker)

Terminal

```
docker-compose up --build
```

Docker Compose starts PostgreSQL and the FastAPI server together.

Terminal — run the Dagster pipeline inside the container

```
docker-compose exec api dagster job execute -f orchestration/etl_job.py -j etl_pipeline
```

Where to Find the Data

Location	Contents
Terminal output (run_demo.py)	Step-by-step printed summaries, region table, A/B results table
http://localhost:8000/docs	Swagger UI — interactive API documentation and live testing
http://localhost:8000/api/v1/regions	JSON — 5 regional cost aggregates
http://localhost:8000/api/v1/experiments	JSON — all pairwise A/B test results
cost_of_living_demo.db (SQLite)	Local database with 3 tables — open with DB Browser for SQLite
data/staging/ (Parquet)	Raw staged data after ingest step (PySpark run only)
data/processed/ (Parquet)	Cleaned, enriched, region-partitioned data (PySpark run only)